

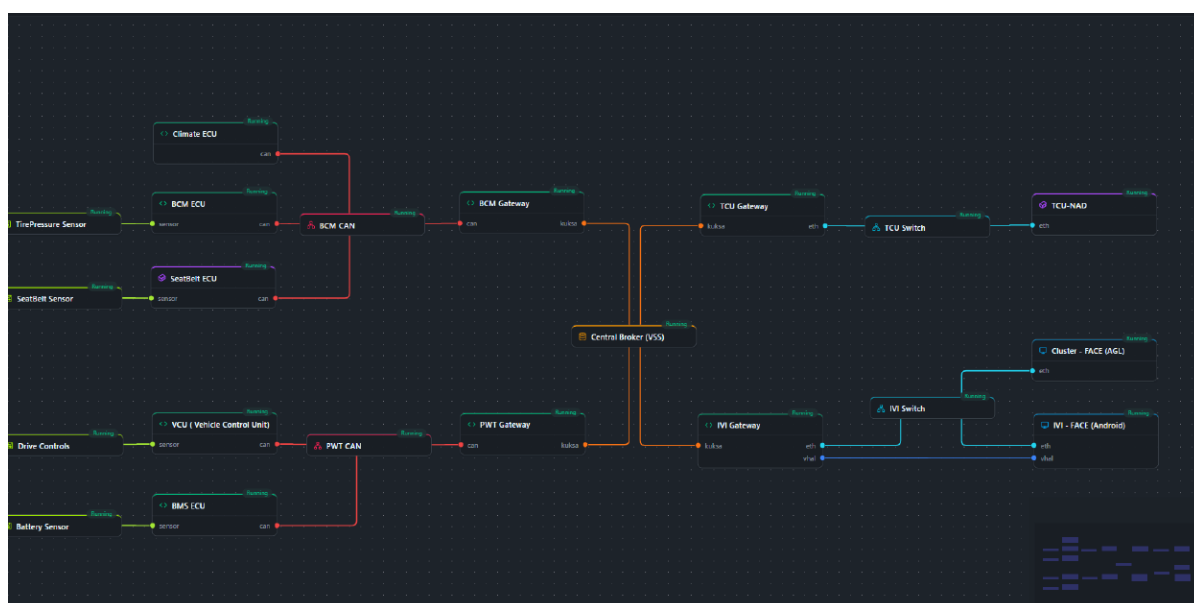
Use case V2X — tư vấn kiến trúc & môi trường

Từ: BTC — FPT Automotive SDV Hackathon · **Gửi:** đội thi use case V2X (cảnh báo chướng ngại ngoài tầm nhìn A-B-C) · **Ngày:** 09/07/2026

1. CarSky — mô hình cần nắm trước khi vẽ blueprint

Vài khái niệm nền của platform để team thiết kế đúng ngay từ đầu:

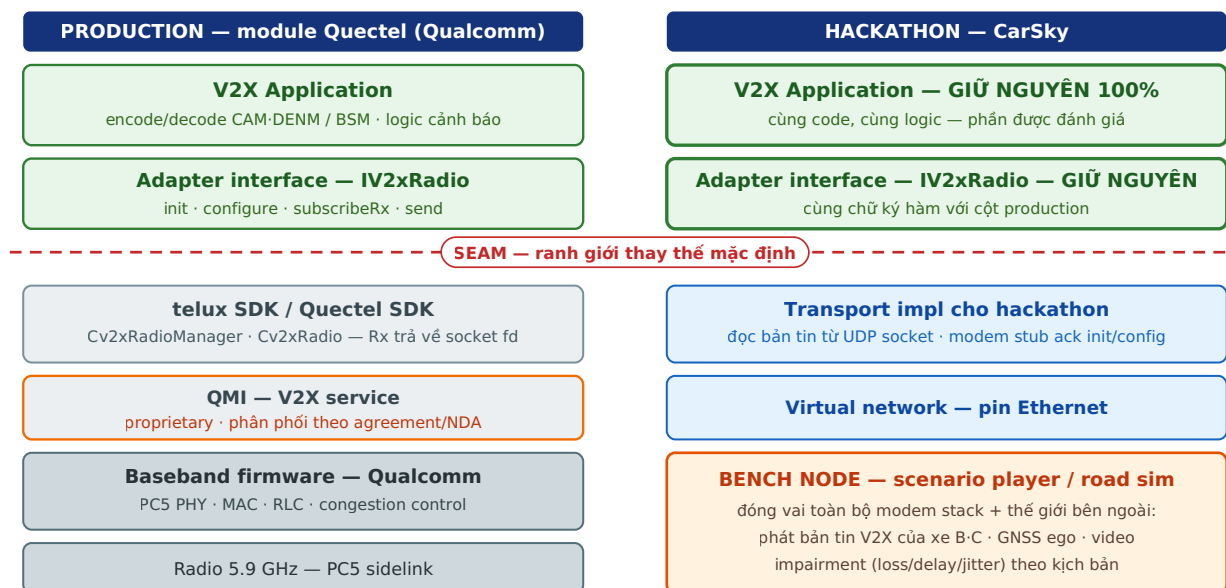
- **Blueprint & node:** 1 xe = 1 blueprint; 1 node = 1 ECU. Node là container — team đóng gói software bất kỳ thành node (Linux application, image OS đầy đủ...). Deploy blueprint là dựng cả chiếc xe ảo chạy trên cloud, thao tác hoàn toàn qua browser.
- **Node có sẵn vs node tự build:** platform có sẵn một số node như **IVI (Android Automotive / AGL)** — head unit ảo đầy đủ, màn hình tương tác trực tiếp trên browser. Ngoài ra là custom node: bất kỳ container nào team tự build. Có thêm **script node** dùng để sinh frame hoặc inject lỗi theo kịch bản.
- **Pin:** các node nối nhau bằng pin — CAN, Ethernet, video... Network ảo giữa các node do platform quản lý.
- **Signal inspector:** xem và inject tín hiệu real-time — hữu ích cho cả debug lẫn trình diễn trước jury.
- **Mức ảo hóa chọn theo từng node:** trong cùng một blueprint, mỗi node có thể ở mức khác nhau — từ **mock** (script / replay data) → **app-level** (production application code chạy trong container Linux) → **full vECU** (image OS đầy đủ như AAOS). Nguyên tắc: chọn mức thấp nhất đủ để kể câu chuyện; chỉ nâng mức ở node nơi trải nghiệm demo của team được thể hiện.
- **Starter pack & sample:** BTC có một road-sim sample (dựng trên MetaDrive) — giá trị tham khảo chính là **code tích hợp với các pin của platform** (phát bản tin lên Ethernet pin, đẩy video lên video pin, feed GNSS...), không phải bản thân sim engine. Team toàn quyền chọn dùng, thay thế, hoặc tự viết.



Một blueprint đang chạy trên platform — mỗi hộp là một node (ECU) đang chạy, các đường nối là pin (CAN / Ethernet / video...).

2. Mapping kiến trúc: production ↔ hackathon

Cách team đặt scope ở mục 2 của câu hỏi (chỉ xử lý application PDU, không tự đóng gói PHY/MAC/RLC) là chính xác — và trong môi trường hackathon, ranh giới lùi thêm đúng một tầng: **cả QMI cũng nằm dưới đường thay thế**.



Trên seam: hai cột identical — effort chuyển đổi rất nhỏ, vì telux Rx trong production vốn đã giao bản tin qua socket.

Dưới seam: phần proprietary của vendor không mang sang được (NDA · không có baseband) — hackathon thay bằng transport giao tiếp với bench.

Đọc diagram theo ba ý:

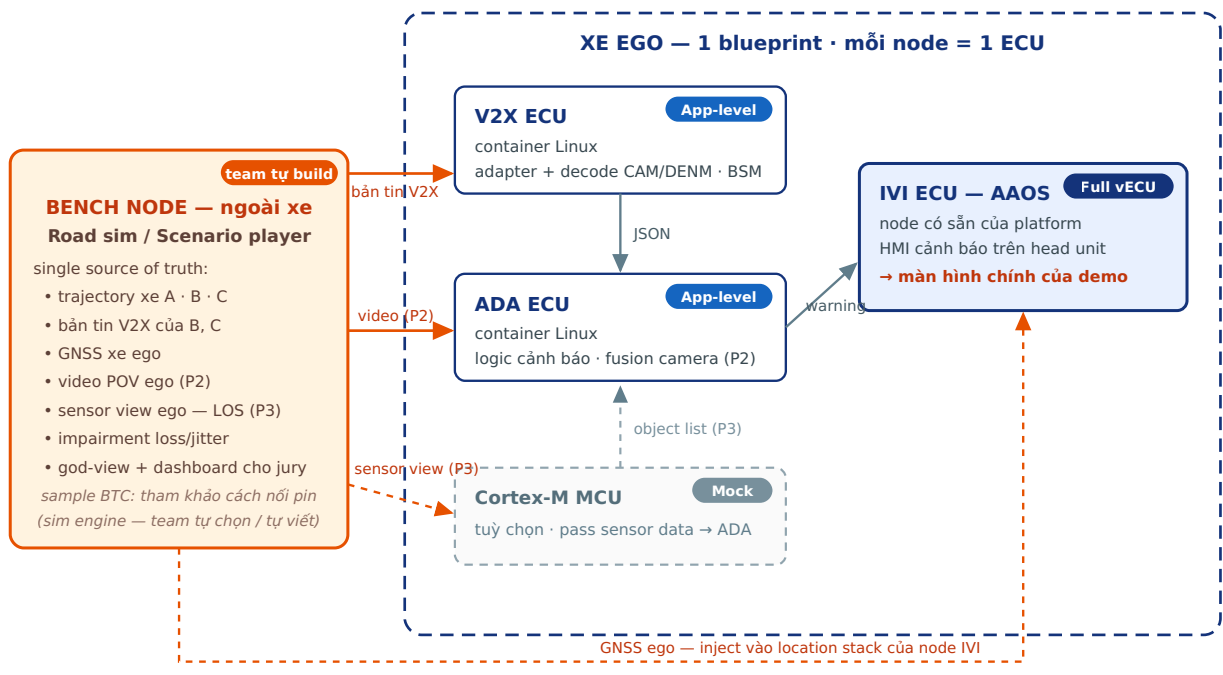
- **Seam mặc định đặt tại adapter interface** trong V2X ECU (`init · configure · subscribeRx · send`). Production: dưới interface là telux/Quectel SDK → QMI → baseband. Hackathon: dưới interface là UDP transport nhận bản tin từ bench node. Đáng chú ý: chính telux cũng giao bản tin cho application **qua socket** (Rx subscription trả về socket fd) — nghĩa là phần nhận của team trong production vốn đã ở dạng "đọc từ socket"; chuyển sang hackathon gần như chỉ đổi phần init/config.
- **Quá trình config** team muốn mô phỏng: một **modem stub** nhỏ trong node V2X ECU nhận đúng trình tự init/configure/subscribe của team và ack (có thể fail-inject để demo error handling); hoặc mở control channel sang bench node nếu muốn config ảnh hưởng luồng phát.
- **Lưu ý về license:** phần client API của telux có tài liệu và mã nguồn công khai, nhưng stack chạy được bên dưới (QMI V2X service, thành phần modem trong BSP) và bộ SDK do module vendor bàn giao vẫn thuộc agreement với Qualcomm/Quectel — và code sản phẩm hiện tại của team thuộc điều khoản NDA giữa công ty team với vendor. Phần code nào link tới các thành phần đó thì để lại phía sau ranh giới thay thế — **không đưa vào repo bài thi**.

Hai mức đầu tư cho phần thay thế modem: (a) **adapter mỏng của team** — phương án nhanh, khuyến nghị mặc định: ít công nhất, đủ cho demo, phù hợp khi team muốn dồn effort cho trải nghiệm. (b) **Ảo hóa telux — porting vào môi trường container** — áp dụng khi code production của team viết trên telux (Telematics SDK của Qualcomm; với stack Quectel/QMI thì nhiều khả năng là vậy — team tự xác nhận trước). Vì mã nguồn client SDK telux là mở, bản chất công việc không phải viết một backend mô phỏng, mà là **porting**: build lại telux cho môi trường container, sửa các lớp phụ thuộc hardware (những điểm telux gọi xuống modem/QMI và thành phần vendor) thành phần hiện thực hướng về bench node, và xử lý các thành phần chưa tương thích giữa môi trường module nhúng và container. Phần API và logic phía trên của telux được tái sử dụng nguyên bản, nên hành vi

(lifecycle/readiness, listener semantics, error model) đúng theo thiết kế gốc — code production của team chạy nguyên vẹn, không sửa dòng nào. Lưu ý: môi trường hiện chưa có sẵn telux được ảo hóa — platform mới áp dụng pattern tương tự cho TelAF (middleware thật compile cho container, lớp HAL được stub); phần ảo hóa TelAF này BTC có thể cung cấp làm tài liệu tham khảo nếu team cần. Việc porting telux vẫn do team tự thực hiện toàn bộ — và cũng chính vì vậy nó là một SKU riêng có giá trị thực. Scope tỷ lệ với số subsystem app sử dụng — bước đầu tiên là kiểm kê: thường tối thiểu gồm cv2x, location và common, vì app V2X production thường lấy GNSS qua chính telux. Dù vậy, khuyến nghị của BTC vẫn là cân nhắc kỹ trước khi nhận SKU này: phương án (b) kể câu chuyện hạ tầng/virtualization, trong khi điểm nhấn của use case (mục 5) nằm ở trải nghiệm trên xe — tránh để hạ tầng chiếm mất quỹ thời gian của phần được trình diễn.

3. Blueprint tham khảo & mức ảo hóa từng node

Blueprint dưới đây là phương án tham khảo do BTC phác thảo từ thiết kế ECU của team, nhằm minh họa cách ánh xạ lên platform — không phải ràng buộc. Phần thuộc mô hình của môi trường (mỗi node là một ECU, một blueprint là một xe, các node nối nhau bằng pin) là cố định; còn cấu trúc node cụ thể, vai trò và mức ảo hóa của từng node hoàn toàn do team quyết định theo đề án của mình.



Blueprint tham khảo: xe ego 3 node thật + 1 node MCU mock tùy chọn; bench node đứng ngoài xe — modem stack, sensor stimulus, toàn bộ "thế giới bên ngoài", kèm màn hình quan sát cho jury (god-view / dashboard).

Node	Vai trò trong demo	Mức đề xuất	Ghi chú
IVI — AAOS (node có sẵn)	HMI hiển thị cảnh báo — màn hình chính của demo	Full vECU	GNSS inject từ nguồn kịch bản; hướng dẫn interface cụ thể đi kèm starter pack
V2X ECU	nhận + decode bản tin (CAM/DENM hoặc BSM), đẩy JSON cho ADA — theo ghi chú thiết kế của team	App-level (container Linux)	lib open-source: Vanetza (ETSI) hoặc asn1c cho SAE J2735
ADA ECU			

	logic cảnh báo "C khuất sau B"; fusion camera ở phase 2	App-level (container Linux)	video: đọc file trực tiếp, hoặc nhận stream qua video pin
Cortex-M MCU (tuỳ chọn)	nhận sensor data từ bench, chuyển tiếp cho ADA	Mock — node riêng, hoặc lược bỏ	radar/lidar thuộc về bench: sensor data synthesize từ kịch bản theo LOS — xem lưu ý dưới và mục 4
Bench (ngoài blueprint xe)	scenario player / road sim — nguồn dữ liệu duy nhất	Team tự build (container)	sample của BTC: tham khảo cách nối pin; sim engine team tự chọn / tự viết

Về nhánh AUTOSAR (Cortex-M MCU + radar/lidar qua CAN_FD): khuyến nghị rõ ràng của BTC — **loại AUTOSAR khỏi phạm vi hackathon**. Ba lý do: (1) môi trường **không cung cấp sẵn AUTOSAR Classic stack**; muốn chạy thật, team tự chuẩn bị stack và license — riêng việc này có thể tốn hơn cả quỹ thời gian còn lại của cuộc thi; (2) chi phí dựng một node Classic AUTOSAR (cấu hình BSW, generate RTE, tích hợp) tính bằng ngày, trong khi trên màn hình demo nó không tạo ra khác biệt nào nhìn thấy được — object list đến từ SWC hay từ một script mock là không phân biệt được — và việc biết dùng AUTOSAR không quá quan trọng so với câu chuyện team đang muốn thể hiện; (3) mỗi giờ đầu tư vào BSW là một giờ bị rút khỏi scenario layer và trải nghiệm trên IVI — phần được đánh giá. Về mô hình: **radar/lidar không phải node của xe** — sensor là điểm tiếp xúc với thế giới vật lý, và trong môi trường ảo, thế giới đó nằm ở bench; dữ liệu sensor vì vậy do bench synthesize từ kịch bản. Cortex-M có hai lựa chọn: giữ làm một node mock riêng trong xe (nhận sensor data từ bench, chuyển tiếp cho ADA — blueprint giữ đúng hình dạng topology sản phẩm), hoặc lược bỏ hoàn toàn — bench inject trực tiếp object list cho ADA.

Packet loss / jitter: khuyến nghị implement ngay trong scenario player (drop / delay / jitter bản tin theo config trước khi phát) — team chủ động hoàn toàn, kịch bản lặp lại chính xác từng lần chạy.

GPU cho ADA ECU (phase 2): GPU trên server chính là tài nguyên chia sẻ giữa nhiều đội — budget hạn chế. Trước khi đăng ký GPU, team nên đánh giá xem demo có thật sự cần chạy vision model không: câu chuyện fusion ("camera thấy B, V2X biết C") vẫn đứng vững khi video chỉ để hiển thị còn detections lấy từ scenario (mục 4) — phương án này không cần GPU. Nhìn theo cấu trúc đề án: khối nhận diện road intelligence từ camera (phát hiện xe, chúng ngại từ video) tự nó là một SKU — chỉ nên đầu tư khi nó nằm trong cấu trúc SKU của đề án (mục 5). Điểm thuận lợi về kiến trúc: khối này xuất ra cùng interface object list với sensor view mock từ bench (mục 4), nên team có thể bắt đầu bằng mock và swap khối perception thật vào sau mà không thay đổi logic ADA. Nếu quyết định làm và cần xử lý ảnh thật, thang lựa chọn theo tải: model nhẹ chạy CPU ở độ phân giải/fps thấp → GPU trên server chính, kèm dự trù cấu hình cụ thể (model, VRAM, fps) thông báo BTC sớm để sắp xếp tài nguyên → nếu tải vượt budget chung, ADA ECU tách thành **external node** chạy trên server riêng — và khi đó bench node đi cùng sang server đó (nguồn video phải nằm cạnh nơi tiêu thụ video), cả hai vẫn nối vào blueprint qua cơ chế external node của platform.

Biến thể nhiều xe thật (tuỳ chọn): platform không giới hạn ở một xe — team có thể deploy nhiều instance của blueprint thành nhiều xe chạy song song, nối với nhau qua một service trung gian; tự nhiên nhất là chính road sim / scenario service đóng vai hub chung mà các xe cùng join làm ground truth. BTC nêu để team biết lựa chọn này tồn tại, kèm đánh giá thẳng thắn: so với 1 xe ego đầy đủ in-car + các xe khác mock trên road sim, phương án nhiều xe thật nhân chi phí theo số xe (tài nguyên, công dụng, và điểm hồng khi demo live) trong khi giá trị nhìn thấy được thường không tăng tương ứng — jury tại một thời điểm chỉ theo dõi một màn hình IVI, còn god-view thì hiển thị mọi xe như nhau dù xe đó thật hay mock. Phương án này chỉ đáng cân nhắc khi điểm nhấn của team (mục 5) bắt buộc có từ hai trải nghiệm in-car đồng thời trở lên — ví dụ hai driver trên hai IVI phản ứng lẫn nhau. Kịch

bản multi-car join chung road sim vì vậy để team tự cân nhắc; khuyến nghị mặc định của BTC vẫn là mô hình 1 ego ở trên.

4. Scenario layer — nguồn dữ liệu duy nhất, và là chỗ tạo khác biệt

Trong môi trường ảo, dữ liệu sensor luôn phải được mock; câu hỏi thực sự không phải "mock hay không" mà là **dữ liệu mock lấy từ nguồn nào để mọi thành phần nhất quán với nhau**. Câu trả lời: một road simulator / scenario player làm single source of truth. Team dựng kịch bản A-B-C (xe C khuất sau xe B) trong sim; từ cùng một nguồn đó xuất ra:

- (a) trajectory của B và C → đóng gói thành bản tin V2X phát cho xe ego;
- (b) trajectory của ego → GNSS inject cho node IVI;
- (c) video POV của ego khớp kịch bản (phase 2);
- (d) sensor view của ego — object list những gì radar/lidar "thấy" theo tầm nhìn (phase 3).

Riêng (d) có một chi tiết quyết định: sensor view phải được **lọc theo line-of-sight** từ ground truth — radar/lidar của ego chỉ "thấy" B, **không thấy C**. Mock naive phát cả C vào sensor sẽ làm mất hiệu lực tiền đề của use case (C phải chỉ đến được ego qua V2X). Bộ lọc LOS này cần hình học ground truth nên chỉ làm được ở scenario layer — thêm một lý do dữ liệu sensor thuộc về bench, không phải một node trong xe.

Vì cùng một nguồn, vị trí trên bản đồ, nội dung cảnh báo, hình ảnh camera và timeline luôn khớp — demo nhất quán và chạy lại được y hệt trước jury. Về công cụ: BTC có sample road sim dựng trên **MetaDrive** — nên dùng nó như tài liệu tham khảo về **cách giao tiếp với các pin** (Ethernet, video, CAN...) trong blueprint, hơn là một sim engine để sử dụng trực tiếp. Renderer 3D của MetaDrive thuộc thế hệ cũ, hình ảnh và hiệu năng render không phù hợp cho demo; nếu dùng, chạy **chế độ 2D (top-down)** — góc nhìn này lại rất phù hợp làm god-view cho màn hình split khi trình diễn (jury thấy cả xe C bị che khuất, cạnh màn hình IVI của ego). Team toàn quyền thay bằng sim khác phù hợp hơn, hoặc tối giản bằng CSV replay tự viết — kịch bản A-B-C của use case này hoàn toàn dựng được bằng trajectory tính tay. Đầu tư vào tầng kịch bản này chính là yếu tố tạo khác biệt về chất lượng demo giữa các đội.

5. Đóng gói đề án — điểm nhấn & cấu trúc SKU

Jury đánh giá một đề án, không đánh giá một checklist công nghệ. Trước khi phân bổ effort, team nên chốt hai câu trả lời và dùng chúng làm trục cho mọi quyết định còn lại:

- **Điểm nhấn — khoảnh khắc jury cần ghi nhớ:** nếu jury chỉ nhớ đúng một điều về đề án, đó là gì? Với use case này, ứng viên tự nhiên: khoảnh khắc IVI cảnh báo xe C **trước khi** camera và mắt người kịp nhìn thấy — "xe biết trước mắt". Split screen god-view (mục 4) đặt cạnh màn hình IVI là cách đơn giản và hiệu quả nhất để khoảnh khắc đó không bị bỏ lỡ. Mọi thành phần khác nên được thiết kế xoay quanh nó.
- **Cấu trúc SKU — đề án gồm những khối nào:** gọi tên được các khối cấu thành, và khối nào là tài sản tái sử dụng. Một cách cất gởi ý cho đề án này: (a) **V2X application** — adapter + decode + logic cảnh báo, phần mang thăng lên sản phẩm thật; (b) **scenario layer** — nguồn kịch bản và dữ liệu, tái sử dụng cho mọi use case V2X về sau chứ không riêng demo này; (c) **HMI cảnh báo** trên IVI. Trình bày theo cấu trúc này cho jury thấy tầm vóc: một sản phẩm có đường phát triển tiếp, không phải demo dùng một lần. Các khối tùy chọn nêu ở mục khác — telux ảo hóa (mục 2), khối perception từ camera (mục 3), dashboard V2N (mục 7) — nếu team chọn làm, cũng được tính là SKU riêng trong cấu trúc này.

Ngược lại, các **beat triển khai** — chạy được AUTOSAR, dựng bus thật, v.v. — là chi tiết execution có thể bổ sung sau khi sản phẩm đã định hình, và không mang trọng số đáng kể trong đánh giá so với tầm vóc và độ hoàn chỉnh của đề án. Đưa chúng vào phần trình bày chính chỉ làm loãng điểm nhấn.

6. Trả lời câu hỏi đợt 1 — mô phỏng bản tin · QMI · camera

- **(1) "Mô phỏng toàn bộ bản tin V2X gửi từ xe B, các quá trình config"** — Bản tin: bench node phát theo kịch bản (mục 4). Khuyến nghị phát dạng encoded chuẩn (ASN.1 UPER) để V2X ECU decode như thật; use case "B cảnh báo hộ C" khớp **DENM** (ETSI) — dùng Vanetza, hoặc asn1c generate codec cho SAE J2735 BSM. Nếu ưu tiên tốc độ, phát trực tiếp dạng JSON cũng chấp nhận được — đánh đổi độ realism lấy thời gian, team tự cân nhắc. Quá trình config: modem stub (mục 2).
- **(2) "Chỉ xử lý bóc tách application PDU / gọi interface setup như QMI của Qualcomm"** — Đúng hướng. Trong hackathon, ranh giới mặc định đặt tại adapter thay vì QMI (hai mức đầu tư — xem mục 2); effort chuyển đổi nhỏ vì đường Rx của production vốn đã là socket. Lưu ý license ở mục 2: thành phần proprietary/NDA giữ ngoài repo bài thi.
- **(3) "Camera: xử lý live feed hay tạm chấp nhận 1 file video"** — Nguồn video: (i) video chuẩn bị sẵn, nội dung khớp kịch bản (cảnh xe B che phía trước) — khuyến nghị mặc định; hoặc (ii) render POV trực tiếp từ road sim — khi đó cần chọn loại sim có 3D renderer đủ tốt, vì fidelity của hình ảnh render ảnh hưởng trực tiếp đến sai số của tầng perception nếu team làm khối này (mục 3), và rendering 3D là tải nặng — cần tránh ảnh hưởng hiệu năng chung của platform trên server chia sẻ (nếu nặng, cân nhắc hướng external node ở mục 3). Về tích hợp: app đọc file trực tiếp (đơn giản nhất), hoặc stream từ bench node qua video pin để giữ nguyên code path live-ingestion. Camera là điểm cộng cho câu chuyện fusion (camera thấy B, V2X biết C sau B) — không phải critical path — nên giới hạn thời gian đầu tư (timebox).

7. Trả lời câu hỏi đợt 2 — mix output giữa các bài & bản đồ

Về nguyên tắc: đề bài và các output mẫu trong đề mang tính minh họa, gợi mở — không phải danh sách bắt buộc. Team có toàn quyền định nghĩa sản phẩm và output của mình, kể cả dùng chéo tài nguyên giữa các bài; BTC đánh giá theo giá trị và độ hoàn chỉnh của đề án, không theo mức độ khớp với mẫu. Hướng team mô tả (V2X obstruction warning + collision monitoring) vì vậy nằm trọn trong tiêu chí. Điều BTC khuyến nghị điều chỉnh là **thứ bậc giữa các output** — áp dụng đúng nguyên tắc ở mục 5, vì chúng không có giá trị ngang nhau khi đánh giá:

- **Hạng nhất — trải nghiệm trên xe:** obstruction warning trên IVI, và chính view 2D/3D "các xe + obstruction ở vị trí tương quan" mà team dự định xây dựng — đây là HMI pattern thật của xe hiện đại (awareness view), đáng đầu tư nhất trong danh sách. Lưu ý kỹ thuật: view này **không cần bản đồ** — nó render từ geometry tương đối (vị trí ego/B/C lấy từ bản tin V2X + GNSS), tự vẽ canvas/scene là đủ.
- **Hạng hai — Live Map Dashboard** (vị trí + trạng thái risk real-time): công cụ trình diễn tốt, đặt cạnh màn hình IVI, cho jury thấy cả xe bị che khuất; nhưng đây là góc nhìn của người quan sát, không phải của driver. Nên có, nhưng ở vai trò hỗ trợ trình diễn — không nên trở thành sản phẩm chính. Cách dựng đủ dùng: một trang web serve từ chính một container node trong blueprint (bench là ứng viên tự nhiên vì đã giữ ground truth; ADA đẩy trạng thái risk sang), mở bằng browser khi demo — không cần hạ tầng riêng. Dashboard chỉ trở thành một SKU thực sự khi có người dùng không phải jury (fleet operator, trung tâm điều hành) — team muốn theo hướng đó thì định vị nó là một khối V2N riêng trong cấu trúc SKU (mục 5) và mô tả rõ người dùng của nó.

- **TTC là engine bên trong:** tính TTC → ngưỡng → trạng thái risk hiển thị lên IVI/dashboard. Stream log và collision event list là bằng chứng kỹ thuật chứng minh logic đúng — thuộc phần validation/phụ lục, không tính là output người dùng.

Một nguyên tắc để team tự kiểm tra khi phân loại output: *sản phẩm là thứ driver nhìn thấy trên xe; dashboard là thứ giúp jury nhìn thấy sản phẩm; log là thứ chứng minh sản phẩm đúng.*

Về bản đồ: bản đồ trong môi trường demo của BTC chạy trên dịch vụ commercial theo license của BTC, nên **BTC không cung cấp bản đồ hay map key cho đội thi**. Team cần map thì tự chủ nguồn, và OpenStreetMap là lựa chọn phù hợp:

- **Trên Android / AAOS:** image AAOS ảo không có Google services nên Google Maps không chạy; renderer cho OSM thì có sẵn — **MapLibre Native** (vector tiles, style hiện đại) hoặc **osmdroid** (raster, đơn giản, không cần key). Cả hai là thư viện Android thuần, không phụ thuộc GMS.
- **Nguồn tile:** free tier của các provider (OpenFreeMap, MapTiler...) với token riêng của team theo điều khoản của họ, hoặc **tile/PMTiles offline đóng gói sẵn theo khu vực kịch bản** — demo không phụ thuộc mạng lẫn key; tránh gọi thẳng tile server công cộng của OSM từ trong app (usage policy). Dashboard web dùng MapLibre GL JS với cùng nguồn tile.
- **Điều kiện để map nền có nghĩa:** kịch bản phải đặt trên tọa độ thật (một giao lộ cụ thể) — khi đó GNSS của ego cũng là tọa độ thật và mọi lớp hiển thị tự khớp (mục 4). Nếu không, dashboard sử dụng trực tiếp top-down view của road sim, không cần map nền.

8. Lộ trình khuyến nghị

P1 — bench → V2X ECU (decode) → ADA (logic "C khuất sau B") → IVI hiển thị cảnh báo. Chuỗi end-to-end này tự nó đã là một demo hoàn chỉnh.

P2 — thêm video + fusion tầm nhìn (camera thấy B, V2X biết C).

P3 — sensor view từ bench (object list theo LOS — qua node MCU mock hoặc inject trực tiếp cho ADA); demo degradation (cảnh báo vẫn đúng khi loss/jitter tăng) nếu còn thời gian.

Mỗi phase demo được độc lập — trượt phase sau không kéo sập phase trước.

Tài liệu tư vấn của BTC trong khuôn khổ FPT Automotive SDV Hackathon. Thông tin platform trong tài liệu ở mức giới thiệu cho đội thi; chi tiết kỹ thuật cụ thể sẽ cung cấp qua starter pack và các phản hồi tiếp theo của BTC.